

GitHub Learning Portfolio

Machine Learning Model Assignment

Car Damage Image Classifier using Teachable Machine

Student Rameswar	Date Submitted March 15, 2026	Tool Used Google Teachable Machine
----------------------------	---	--

Learning Objectives

After completing this assignment, I can:

- Create a basic machine learning model using an online application.
- Describe any challenges faced when designing, testing, or implementing the model.

Question A — How Teachable Machine Uses Machine Learning

Teachable Machine is a web-based tool developed by Google that makes machine learning accessible to anyone, regardless of coding background. It uses a process called transfer learning, which is a technique where a model that has already been trained on a very large dataset (in this case, MobileNet — a lightweight convolutional neural network pre-trained on millions of images) is adapted to recognize a new, user-defined set of categories.

Here is how the process works step by step:

- **Data Collection:** The user provides sample images (or audio/poses) for each class by uploading files or capturing them through a webcam. These samples become the training data.
- **Feature Extraction:** Instead of training from scratch, Teachable Machine uses the existing MobileNet backbone to extract meaningful numerical features (embeddings) from every image. This step is instant because MobileNet's weights are frozen — Teachable Machine does not retrain them.
- **Classification Layer Training:** A small, new classification layer is added on top of MobileNet. Only this layer is trained on the user's samples. It learns to map the extracted features to the user-defined class labels using a technique called k-nearest neighbors (KNN) or a lightweight neural network head, depending on the version.
- **Prediction:** Once trained, the model takes a new, unseen image, runs it through MobileNet to get its feature vector, then passes that vector through the classification layer to output a probability score for each class. The class with the highest probability is the model's prediction.

The key insight behind Teachable Machine is that the hard work of learning general visual features (edges, textures, shapes) has already been done by MobileNet during its original training. The user only needs to teach the model the distinctions that matter for their specific problem. This makes the entire training loop run in seconds inside a browser, with no server required.

Question B — Application Chosen

I chose to build a Car Damage Severity Classifier — an image classification application that automatically categorizes photographs of damaged vehicles into one of three severity levels:

- Minor Damage — small dents, surface scratches, chipped paint with no structural impact.
- Moderate Damage — crumpled body panels, broken headlights or taillights, visibly deformed bumpers.
- Severe Damage — structural damage such as a crushed cabin, deployed airbags, or a vehicle that is considered a total loss.

The model was trained using Google's Teachable Machine (Image Project mode), which is the platform's most capable and easiest mode to work with for photo-based classification. Each class was trained with a minimum of 80 representative images collected from open-source automotive damage photo datasets, ensuring that the model was exposed to a variety of vehicle makes, angles, lighting conditions, and damage patterns.

Question C — Problem the Application Is Intended to Solve

The core problem this application addresses is the slow, expensive, and inconsistent nature of manual vehicle damage assessment in the automobile insurance and repair industry.

Currently, when a vehicle is damaged in an accident, the typical workflow looks like this:

- The vehicle owner files a claim and schedules an in-person inspection.
- A licensed adjuster drives to the vehicle or a repair facility to perform a physical assessment.
- The adjuster manually categorizes the damage and estimates repair costs.
- A written report is generated, often days after the initial incident.

This process has several pain points. First, it is time-consuming: according to industry data, the average claim processing time ranges from 7 to 14 days, largely due to scheduling and travel requirements. Second, assessments can be inconsistent between adjusters, particularly for borderline cases that sit between "moderate" and "severe." Third, it requires physical presence, which is a limitation in remote areas or during high-volume events like natural disasters.

A lightweight AI classifier like this one could serve as a first-pass triage tool: a vehicle owner submits photos through a mobile app immediately after an accident, the model instantly categorizes the damage severity, and the output is used to route the claim appropriately — for example, directing minor damage cases to a self-service repair portal and escalating severe damage cases to a human specialist immediately. This reduces processing time, improves consistency, and allows human adjusters to focus their expertise where it matters most.

Question D — Why I Chose This Application

I chose this project for three interconnected reasons: personal relevance, real-world applicability, and the strong technical fit between the problem and Teachable Machine's capabilities.

On a personal level, I experienced a frustrating multi-week wait during an insurance claim after a minor collision, where the damage to my vehicle was clearly cosmetic but still required a full in-person adjuster visit. That experience planted a question in my mind: why can't this initial step be automated? Building this classifier was a way of turning that frustration into a learning opportunity.

From a professional standpoint, the insurance technology (InsurTech) sector is one of the fastest-growing areas applying AI to legacy processes. Companies such as Tractable and Snapsheet have raised hundreds of millions of dollars building AI-powered claims automation tools. Understanding how even a basic version of this technology works gives me insight into a domain where machine learning is actively creating business value.

Finally, the problem is a clean three-class image classification task, which is exactly the type of problem Teachable Machine is built to handle. The visual differences between minor, moderate, and severe damage are substantial enough that a transfer-learning-based model can learn them from a relatively small dataset. This made the project achievable within the scope of the assignment while still being meaningful and non-trivial.

Question E — URL to Exported Model / Application

Teachable Machine Exported Model

Public model URL <https://teachablemachine.withgoogle.com/models/FvQR7b-Qe/>

Challenges Faced During Design, Testing, and Implementation

Several meaningful challenges arose during the development of this model:

Challenge 1 — Class Boundary Ambiguity: The line between “moderate” and “severe” damage is not always clear-cut, even to human experts. During initial testing, the model frequently misclassified heavily crumpled hoods as “severe” when the structural integrity of the passenger cabin was still intact. To address this, I revisited the class definitions, added more boundary-case images to the moderate class, and retrained the model. This iteration reduced the moderate-vs-severe confusion rate noticeably.

Challenge 2 — Dataset Imbalance: Truly totaled vehicles are less common in photo datasets than minor fender-benders. My initial dataset had roughly 120 minor, 90 moderate, and 45 severe images — a significant imbalance. Teachable Machine does not offer explicit class-weighting controls, so I compensated by adding more severe damage images and using data augmentation strategies (mirroring, cropping) within the tool to artificially expand the severe class.

Challenge 3 — Lighting and Background Variation: Some test images taken in dim parking garages or at night caused the model to underperform, as the training set was dominated by well-lit outdoor photos. Incorporating a subset of low-light images into each class improved robustness, though some sensitivity to extreme lighting conditions remains.

Challenge 4 — Model Export and Deployment: Understanding the difference between downloading the model files (TensorFlow.js format) versus using the shareable URL was initially confusing. The exported model consists of a metadata.json file, a model.json file, and several weight shard files, which require a web server to serve correctly. Using the hosted shareable URL was the more straightforward path for this assignment.

Personal & Professional Connections

This project deepened my understanding of several concepts that connect directly to my broader studies and career interests:

- **Transfer Learning in Practice:** Reading about transfer learning in a textbook is very different from watching MobileNet embeddings power a real classifier I built in minutes. The experience made the concept concrete and gave me an intuition for when and why pre-trained models are preferable to training from scratch.
- **Human-in-the-Loop Systems:** The challenges I encountered — especially boundary cases between damage classes — reinforced why fully automated AI decisions in high-stakes domains like insurance are risky. A well-designed system keeps a human in the loop for ambiguous predictions, using confidence thresholds to route uncertain cases to manual review.
- **Data Quality Over Quantity:** The dataset imbalance challenge was a hands-on lesson in why “more data” is not always the answer. A smaller, well-curated and balanced dataset produced better results than a large imbalanced one.
- **InsurTech Industry Awareness:** Researching the problem domain for this project introduced me to companies like Tractable, which uses deep learning to automate claims assessments at scale, and Mitchell International, which provides AI-powered repair estimation tools. Understanding the competitive landscape makes the technical work feel grounded in real-world context.

Looking forward, I am interested in exploring how this classifier could be extended: for instance, integrating it with a cost-estimation model that uses the damage severity prediction as one of several inputs (along with vehicle make, model, and year) to produce an automated repair cost range. That kind of multi-model pipeline is where real-world InsurTech applications live, and this assignment gave me a solid foundation to start thinking about it.

Additional Resources & References

The following resources informed and enriched this portfolio entry:

- **Google Teachable Machine** — teachablemachine.withgoogle.com
- **MobileNet Paper (Howard et al., 2017)** — arxiv.org/abs/1704.04861
- **Tractable AI — AI-powered claims automation** — tractable.ai
- **Google ML Crash Course: Transfer Learning** — developers.google.com/machine-learning/crash-course
- **TensorFlow.js Model Deployment Guide** — www.tensorflow.org/js/guide/save_load

Self-Assessment Against Active Learning Rubric

Criterion	Evidence in This Entry	Target Level
Knowledge	Accurate, complete explanation of transfer learning, MobileNet, KNN classification, dataset strategies, and deployment formats.	Exemplary (4)
Inquiry	Step-by-step technical rationale for every design decision; additional links to MobileNet paper, Tractable, TF.js guide, and ML Crash Course.	Exemplary (4)

Criterion	Evidence in This Entry	Target Level
Communication — Audience & Meaning	Clear structure with section headings; written for a reader with basic technical literacy; engaging narrative connecting personal experience to industry context.	Exemplary (4)
Communication — Conventions	Technical terminology used correctly (transfer learning, embeddings, KNN, class imbalance, confidence thresholds); proofread for grammar.	Exemplary (4)
Making Connections	Personal insurance experience, InsurTech industry landscape (Tractable, Snapsheet, Mitchell), and future multi-model pipeline ideas.	Exemplary (4)
TOTAL	<i>Exemplary, Exceeds Expectations</i>	20 / 20